Linux Processer

interrupts.c

Studieguid med uppgifter

Programmering av inbyggda system · Jensen YH IoTS25

Mikael Wallin · Unix Tooling

1. Setup — Så kör du på PC och Pi Zero 2W

Hur C-filer kompileras

Alla uppgifter är skrivna i C. Du kompilerar med gcc och kör den skapade binären.

Arbetsflöde (gäller på BÅDE PC/Ubuntu och Pi Zero 2W):

1. Skapa en fil, t.ex.: interrupts.c
2. Kompilera: gcc -o interrupts interrupts.c
3. Kör: ./interrupts
4. Öppna andra terminal: kill -USR1 <PID> # skicka signal

På din PC (Ubuntu / WSL)

Installera build-tools om de saknas:

```bash
sudo apt update
sudo apt install build-essential
```

På Raspberry Pi Zero 2W

Samma kommandon. SSH in först, sedan kompilera och kör direkt på Pi:n:

```bash
# Från din PC — logga in på Pi
ssh <dittnamn>@<pi-ip>

# På Pi — installera gcc om det saknas
sudo apt install gcc

# Kopiera din fil till Pi (kör från PC, inte Pi)
scp interrupts.c <dittnamn>@<pi-ip>:~/

# Eller skapa filen direkt på Pi med nano
nano interrupts.c
```

Två terminaler — varför behövs det?

Programmet i uppgifterna kör i en oändlig loop. Du kan inte skriva i samma terminal. Lösning:

Terminal 1: Kör programmet ./interrupts
Terminal 2: Skicka signaler kill -USR1 <PID>

Hitta PID: echo $$ (i Terminal 1 INNAN du startar programmet)

eller: ps aux | grep interrupts

2. Teori — Vad är en process?

Program vs Process

Det är lätt att blanda ihop dessa två begrepp:

Begrepp	Förklaring
Program	En fil på disken (t.ex. interrupts.c kompilerat till interrupts). Recept som väntar på att köras.
Process	Programmet i minnet och under körning. Du kan köra samma program 10 gånger = 10 separata processer.

Vad Linux ger varje process

När Linux startar en process skapas en isolerad miljö med:

`PID`	Process ID — ett unikt heltal. Används med kill-kommandot och syns i ps/top.
`Virtuellt minne`	Processen tror att den har hela maskinen för sig själv. Kernel sköter den verkliga mappningen.
`Ägare (user)`	Varje process kör som en användare. Avgör vilka filer den kan läsa/skriva.
`File descriptors`	Handtag till filer, pipes, sockets. Du börjar alltid med stdin(0), stdout(1), stderr(2).
`Working dir`	Mappen processen befinner sig i — vad du ser som 'aktuell katalog'.
`Env-variabler`	Nyckel-värde par ärvda från föräldern: PATH, HOME, LANG etc.

Processträdet — PID 0, 1, 2

Alla processer har en förälder. Det bildar ett träd ända upp till PID 1.

`PID 0`	Swapper / idle. Inte en riktig process — kärnan kör denna kod när inget annat finns att schemalägga. Syns INTE i ps.
`PID 1`	systemd / init. Första riktiga processen. Startar systemet och adopterar föräldralösa processer (zombie-prevention).
`PID 2`	kthreadd. Förälder till alla kernel-trådar (kworker, ksoftirqd, migration). Syns i ps inom hakparenteser: [kworker].

```bash
pstree -p   # visa hela processträdet med PID
```

3. Forking — Hur processer skapas

Linux skapar nya processer via systemanropet fork(). Föräldern kopieras — barnet är en exakt kopia.

fork() gör följande:

- Skapar en ny process (barnet) som är en kopia av föräldern
- Båda körs vidare från samma punkt i koden
- Skillnaden: fork() returnerar barnets PID till föräldern, och 0 till barnet
- Föräldern och barnet är sedan helt oberoende

```c
pid_t child = fork();

if (child == 0) {
    // Denna kod körs BARA av barnet
    printf("Jag är barnet, mitt PID: %d\n", getpid());
} else {
    // Denna kod körs BARA av föräldern
    printf("Jag är föräldern, barnets PID: %d\n", child);
    waitpid(child, NULL, 0);  // Vänta tills barnet avslutar
}
```

Prova i terminalen

Dessa kommandon visar forking i praktiken:

```bash
echo $$          # Visa nuvarande shell:s PID
bash             # Starta ett nytt shell (ett barn-process)
echo $$          # Barnets PID (skiljer sig från föräldern)
echo $PPID       # Förälderns PID (matchar det vi såg ovan)
exit             # Avsluta barnet, tillbaka till föräldern
echo $$          # Förälderns PID igen
```

4. Processtillstånd

En process befinner sig alltid i ett av dessa tillstånd. Du ser det i kolumnen STAT i ps aux:

Tillstånd	Kod	Vad händer?
Running	`R`	Körs aktivt på en CPU-kärna just nu. Endast detta tillstånd använder faktisk CPU.
Sleeping	`S`	Väntar på något — knapptryckning, fil, timer. De flesta processer är här det mesta av tiden.
Uninterruptible Sleep	`D`	Väntar på hårdvaru-I/O. Kan INTE väckas av signal — inte ens SIGKILL. Syns vid hängande NFS-mount.
Stopped	`T`	Pausad, vanligtvis av SIGSTOP eller Ctrl+Z. Återupptas med SIGCONT. Du ser detta i Uppgift 8.
Zombie	`Z`	Processen är klar men föräldern har inte hämtat exit-koden ännu. Tar inget minne, bara ett PID och en exit-kod.

```bash
ps aux | grep interrupts   # Se STAT-kolumnen
```

5. Signaler — Kommunicera med en process

En signal är ett meddelande till en process. Det enda gränssnittet mot ett program som kör i bakgrunden.

Vanliga signaler att känna till

SIGINT (2)	Ctrl+C. Be processen avsluta snällt.
SIGTERM (15)	kill <pid> standard. Be processen avsluta snällt. KAN fångas.
SIGKILL (9)	kill -KILL <pid>. Kernel dödar processen omedelbart. KAN INTE fångas.
SIGSTOP (19)	kill -STOP <pid> eller Ctrl+Z. Pausar processen. KAN INTE fångas.
SIGCONT (18)	kill -CONT <pid>. Återupptar en pausad process.
SIGHUP (1)	Terminalen kopplades bort. Används av daemons för att ladda om config.
SIGUSR1 (10)	Användardefinierad — du bestämmer vad den gör i ditt program.
SIGUSR2 (12)	Användardefinierad — du bestämmer vad den gör i ditt program.
SIGFPE (8)	Kernel skickar vid division med noll.
SIGSEGV (11)	Kernel skickar vid null pointer / felaktig minnesåtkomst.
SIGCHLD (17)	Kernel skickar till förälder när ett barn avslutar.
SIGPIPE (13)	Skrivning till bruten pipe.
SIGALRM (14)	Timer från alarm()-systemanropet.

Skicka signaler från terminalen

```bash
kill -USR1 <pid>        # Skicka SIGUSR1 till en specifik process
kill -USR1 -<pgid>      # Skicka till hela process-gruppen
kill <pid>              # Skickar SIGTERM (standard)
kill -KILL <pid>        # Skickar SIGKILL (nuclear option)
kill -STOP <pid>        # Pausa processen
kill -CONT <pid>        # Återuppta processen
kill -HUP  <pid>        # Skicka SIGHUP

killall interrupts      # Skicka SIGTERM till allt som heter 'interrupts'
pkill interrupts        # Samma sak, mer flexibelt
```

Hantera signaler i C-kod

Du registrerar en handler-funktion som körs när signalen anländer:

```c
#include <signal.h>
#include <stdio.h>
```

```
// Handler-funktionen — måste ha signaturen void f(int sig)
void handle_sigusr1(int sig) {
    printf("[SIGNAL] SIGUSR1 mottagen!\n");
}

int main() {
    // Registrera handleren
    signal(SIGUSR1, handle_sigusr1);

    // Programmet kör vidare...
    while (1) { sleep(1); }
    return 0;
}
```

6. Startkoden — interrupts.c

Alla uppgifter bygger på denna fil. Skapa den, kompilera och testa INNAN du börjar med uppgifterna.

Det här programmet modellerar en långkörande tjänst (som en sensor-daemon på Pi:n).
Det startar, skriver ut sitt PID, och kör sedan en oändlig loop.
Du styr det utifrån med signaler från Terminal 2.

```c
#include <stdio.h>
#include <signal.h>
#include <unistd.h>
#include <stdlib.h>
#include <stdatomic.h>

volatile atomic_int running = 1;   // 1 = kör, 0 = avsluta
volatile atomic_int counter = 0;   // räknare som ökar varje sekund

// Handler för SIGINT (Ctrl+C) — sätt running till 0
void handle_sigint(int sig) {
    printf("\n[INTERRUPT] SIGINT — avslutar snällt...\n");
    running = 0;
}

// Handler för SIGUSR1 — nollställ räknaren
void handle_sigusr1(int sig) {
    counter = 0;
    printf("\n[INTERRUPT] SIGUSR1 — räknare nollställd\n");
}

int main() {
    printf("Startar. PID = %d\n", getpid());
    printf("Skicka SIGUSR1 för att nollställa: kill -USR1 %d\n",
getpid());

    signal(SIGINT,  handle_sigint);
    signal(SIGUSR1, handle_sigusr1);

    while (running) {
        sleep(1);
        counter++;
        printf("Räknare: %d\n", counter);
    }

    printf("Avslutar snällt. Hej då!\n");
    return 0;
}
```

Kompilera och testa

```bash
```

```
gcc -o interrupts interrupts.c

# Terminal 1
./interrupts

# Terminal 2 — hitta PID och testa
ps aux | grep interrupts
kill -USR1 <pid>         # nollställ räknaren
kill -STOP <pid>         # pausa
kill -CONT <pid>         # återuppta
```

7. Uppgifter

Varje uppgift bygger på den föregående. Spara filen och lägg till kod steg för steg.
Kompilera och testa efter VARJE uppgift innan du går vidare.
Referens: https://man7.org/linux/man-pages/man7/signal.7.html

Uppgift 1 — Fånga division med noll (SIGFPE)

Uppgift 1 — Catch a divide by zero

Lägg till en handler för SIGFPE. Inuti main(), efter signal-registreringen, lägg till:

int x = 1 / 0;

Utan handler kraschar programmet. Din handler ska fånga signalen, skriva ut ett meddelande,
och anropa exit(0) istället för att krascha.

OBS: Till skillnad mot SIGINT kan du INTE bara returnera från denna handler.
CPU:n utlöser felet igen direkt. Anropa exit(0) för att avsluta rent.

Lägg till i din kod:

```c
// Handler för division med noll
void handle_sigfpe(int sig) {
    printf("\n[FEL] Division med noll fångad! Avslutar rent.\n");
    exit(0);   // Måste exit() — kan inte bara returnera
}

// I main() — registrera och trigga:
signal(SIGFPE, handle_sigfpe);
int x = 1 / 0;   // Triggar SIGFPE
```

Förväntat resultat:

[FEL] Division med noll fångad! Avslutar rent.

Uppgift 2 — Återhämta sig utan att avsluta (setjmp/longjmp)

Uppgift 2 — Recover from divide by zero without exiting

Utöka Uppgift 1 så att programmet ÖVERLEVER felet och fortsätter köra.
Slå upp setjmp() och longjmp(). Placera setjmp() före divisionen.
Anropa longjmp() från handleren. Programmet ska skriva ut ett recovery-meddelande
och fortsätta sin main-loop som om ingenting hänt.

```c
#include <setjmp.h>

jmp_buf recovery_point;   // Sparar körningspunkten

void handle_sigfpe(int sig) {
    printf("\n[RECOVERY] FPE fångad — hoppar tillbaka!\n");
    longjmp(recovery_point, 1);   // Hoppa tillbaka till setjmp-punkten
}

// I main() — FÖRE den farliga koden:
if (setjmp(recovery_point) == 0) {
    // Förstagångskörning
    int x = 1 / 0;
} else {
    // Hamnar här efter longjmp
    printf("[INFO] Återhämtning lyckad, fortsätter loop...\n");
}
```

Uppgift 3 — Lägg till SIGUSR2 som stats-signal

Uppgift 3 — Add SIGUSR2 as a stats signal

Lägg till en andra user-signal som skriver ut nuvarande räknarvärde UTAN att nollställa.

SIGUSR1 nollställer redan räknaren.

SIGUSR2 ska BARA rapportera värdet.

```c
void handle_sigusr2(int sig) {
    printf("\n[STATS] Nuvarande räknare: %d\n", counter);
}

// I main() — registrera:
signal(SIGUSR2, handle_sigusr2);

// Testa från Terminal 2:
// kill -USR1 <pid>    # nollställ
// kill -USR2 <pid>    # rapportera
```

Uppgift 4 — Lägg till SIGTERM

Uppgift 4 — Add SIGTERM

SIGTERM är vad kill <pid> skickar som standard — inget flagg behövs.

Just nu dör programmet omedelbart vid kill <pid>.

Lägg till en handler som sätter running = 0 precis som SIGINT gör.

Du kan återanvända exakt samma handler-funktion för båda signalerna.

Testa skillnaden:

kill <pid> # SIGTERM — ska avsluta snällt

kill -KILL <pid> # SIGKILL — kan inte fångas, alltid

```c
// Registrera SAMMA handler för både SIGINT och SIGTERM:
signal(SIGINT,  handle_sigint);
signal(SIGTERM, handle_sigint);   // Återanvänd samma funktion!
```

Uppgift 5 — SIGHUP som verbose-toggle

Uppgift 5 — Add SIGHUP as a toggle

Lägg till en handler för SIGHUP som togglar ett verbose-läge.

När verbose är PÅ: skriv extra detalj varje loop-varv.

När verbose är AV: skriv minimal output.

Det här speglar hur riktiga daemons använder SIGHUP för att ladda om config utan omstart.

```c
volatile atomic_int verbose = 0;

void handle_sighup(int sig) {
    verbose ^= 1;   // XOR-toggle: 0->1->0->1...
    printf("\n[INTERRUPT] SIGHUP — verbose %s\n", verbose ? "PÅ" : "AV");
}

// I main() — registrera och använd:
signal(SIGHUP, handle_sighup);

while (running) {
    sleep(1);
    counter++;
    if (verbose) {
        printf("[VERBOSE] Räknare: %d | PID: %d | verbose: %d\n",
               counter, getpid(), (int)verbose);
    } else {
        printf("Räknare: %d\n", counter);
    }
}

// Testa: kill -HUP <pid>
```

Uppgift 6 — Timer med SIGALRM

Uppgift 6 — Add a timer with SIGALRM

Gör så att programmet skickar en signal till sig självt var 5:e sekund — utan kill-kommando.

Anropa alarm(5) i main() INNAN loopen. Återarma den i slutet av handleren.

Skriv ut ett heartbeat-meddelande. Detta är en mjukvarutimer —

samma koncept som ett hårdvarutimer-interrupt på en mikrokontroller!

```c
void handle_sigalrm(int sig) {
    printf("\n[TIMER] 5 sekunder — räknare är %d\n", (int)counter);
    alarm(5);   // Återarma för nästa gång
}

// I main() — registrera och starta timer:
signal(SIGALRM, handle_sigalrm);
alarm(5);   // Starta timer (skickar SIGALRM om 5 sekunder)

// Kopplingen till embedded: alarm() = software timer interrupt
// På Arduino: Timer1.attachInterrupt(handler)  → samma princip!
```

Uppgift 7 — Blockera SIGTERM med sigprocmask()

Uppgift 7 — Block SIGTERM

Använd sigprocmask() för att blockera SIGTERM så att kill <pid> inte har någon effekt.

Signalen köas men levereras inte.

Observera att enda sättet att stoppa programmet utifrån nu är kill -KILL.

Det här läget dig förstå skillnaden mellan:

- Blockera: signalen väntar (köas, levereras senare när du tar bort blockeringen)
- Ignorera: signalen kastas (aldrig levererad)
- Och varför SIGKILL existerar

```c
#include <signal.h>

// Blockera SIGTERM (placera i main() före loopen):
sigset_t mask;
sigemptyset(&mask);             // Skapa tom mask
sigaddset(&mask, SIGTERM);      // Lägg till SIGTERM
sigprocmask(SIG_BLOCK, &mask, NULL);  // Blockera

// Nu: kill <pid>       = INGEN EFFEKT (köas)
// Nu: kill -KILL <pid> = Dör ändå (SIGKILL kan ALDRIG blockeras)
```

Uppgift 8 — Styr processen från ett andra shell

Uppgift 8 — Control the process from a second shell

Kör ditt program i Terminal 1. Öppna Terminal 2 och styr med BARA kill-kommandon.
Inga tangentbordsgenvägar tillåtna.
Arbeta igenom denna sekvens i ordning:

```bash
kill -USR1 <pid>    # nollställ räknaren
kill -USR2 <pid>    # skriv ut stats
kill -STOP <pid>    # frys processen
ps aux | grep interrupts  # → se STAT ändras till T
kill -CONT <pid>    # återuppta
kill -HUP  <pid>    # toggla verbose
kill -TERM <pid>    # snäll shutdown
```

Använd ps, top eller pstree -p i Terminal 2 för att se processtillståndet ändras.
När du skickar SIGSTOP ska du se STAT-kolumnen i ps ändras från S till T.

ps aux | grep interrupts # Kör detta MEDAN programmet är pausat med SIGSTOP

Uppgift 9 — Forka ett barn-process

Uppgift 9 — Fork a child process

Lägg till ett fork()-anrop inuti main() INNAN loopen.
Barnet ska köra sin egna räknar-loop.
Föräldern ska bevaka barnet med waitpid() och skriva ut ett meddelande när det avslutar.

Skicka SIGTERM till föräldern och observera: fortsätter barnet att köra?
Slå sedan upp process-grupper och kill(-pgid) för att förstå
hur man signalerar en hel family på en gång.

```c
pid_t child = fork();

if (child == 0) {
    // BARNETS kod
    printf("[BARN] Startar. PID=%d, PPID=%d\n", getpid(), getppid());
    int c = 0;
    while (1) {
        sleep(1);
        printf("[BARN] räknare: %d\n", c++);
    }
    exit(0);

} else if (child > 0) {
    // FÖRÄLDERNS kod
```

```
    printf("[FÖRÄLDER] Barnets PID: %d\n", child);
    // ... förälderns loop ...
    int status;
    waitpid(child, &status, 0);
    printf("[FÖRÄLDER] Barnet avslutade.\n");

} else {
    // fork() misslyckades
    perror("fork misslyckades");
    exit(1);
}
```

Uppgift 10 — Byt signal() mot sigaction()

Uppgift 10 — Replace signal() with sigaction()

Skriv om ALLA dina signal-registreringar med sigaction() istället för signal().
signal() är inkonsekvent mellan plattformar (beter sig olika på Linux vs macOS).
sigaction() är det korrekta POSIX-API:et.

sigaction() låter dig:

- Inspektera den gamla handleren innan du ersätter den
- Kontrollera om handleren återställs till default efter att den utlösts
- Specificera SA_RESTART för att automatiskt starta om avbrutna syscalls

Det här är versionen du skulle använda i produktionskod.

```c
#include <signal.h>

// Ersätt: signal(SIGINT, handle_sigint);
// Med:    sigaction(SIGINT, &sa, NULL);

struct sigaction sa;
sa.sa_handler = handle_sigint;     // Din handler-funktion
sigemptyset(&sa.sa_mask);          // Inga extra signaler blockerade
under hantering
sa.sa_flags = SA_RESTART;          // Återstarta avbrutna syscalls
automatiskt
sigaction(SIGINT, &sa, NULL);      // Registrera (NULL = spara inte
gammal handler)

// Gör samma sak för ALLA dina handlers:
// SIGTERM, SIGUSR1, SIGUSR2, SIGHUP, SIGALRM, SIGFPE
```

8. Snabbreferens — Signaler och kommandon

De viktigaste signalerna

SIGINT (2)	Ctrl+C — avsluta snällt. KAN fångas.
SIGTERM (15)	kill <pid> standard — avsluta snällt. KAN fångas.
SIGKILL (9)	kill -KILL — dödar omedelbart. KAN INTE fångas eller blockeras.
SIGSTOP (19)	Pausa. KAN INTE fångas. Tillstånd → T
SIGCONT (18)	Återuppta pausad process.
SIGHUP (1)	Terminal kopplades bort. Config-reload i daemons.
SIGUSR1 (10)	Fri för egna ändamål.
SIGUSR2 (12)	Fri för egna ändamål.
SIGFPE (8)	Division med noll. Kernel skickar automatiskt.
SIGSEGV (11)	Ogiltig minnesåtkomst. Kernel skickar automatiskt.
SIGALRM (14)	alarm()-timer löpte ut. Kernel skickar automatiskt.
SIGCHLD (17)	Barn-process avslutade. Kernel skickar till förälder.
SIGPIPE (13)	Skrivning till bruten pipe. Kernel skickar automatiskt.

kill-kommandon

```bash
kill <pid>              # SIGTERM (standard)
kill -KILL <pid>        # SIGKILL
kill -STOP <pid>        # Pausa
kill -CONT <pid>        # Återuppta
kill -USR1 <pid>        # SIGUSR1
kill -USR2 <pid>        # SIGUSR2
kill -HUP  <pid>        # SIGHUP
kill -TERM <pid>        # SIGTERM (explicit)
kill -USR1 -<pgid>      # Till hela process-gruppen
killall interrupts      # Via process-namn
pkill interrupts        # Via namn, mer flexibelt
```

Användbara kommandon för felsökning

```bash
echo $$                 # Aktuella shells PID
ps aux                  # Lista alla processer
ps aux | grep interrupts  # Filtrera en specifik process
pstree -p               # Visa processträdet med PID
top                     # Realtids processöversikt (q för avsluta)
```

Man-sidor att bokmärka

https://man7.org/linux/man-pages/man7/signal.7.html — Alla signaler
https://man7.org/linux/man-pages/man2/sigaction.2.html — sigaction()
https://man7.org/linux/man-pages/man2/sigprocmask.2.html — sigprocmask()
https://man7.org/linux/man-pages/man2/waitpid.2.html — waitpid()
https://man7.org/linux/man-pages/man2/alarm.2.html — alarm()
https://man7.org/linux/man-pages/man3/longjmp.3.html — longjmp()

man7.org är den officiella källan. mankier.com är mer nybörjarvänlig.
Slå upp man-sidor direkt i terminalen: man 2 sigaction